11 MAGGIO 2026

Analisi statica dei programmi / Ispezione del codice

MAFFEIS RICCARDO 1085706

Sommario

1. Introduzione 2
2. Metodo di ispezione 2
3. Riepilogo dei 20 punti di controllo 2
4. Analisi dettagliata punto per punto 4
1. Obiettivo dell'analisi 4
2. Struttura generale del programma 4
3. Leggibilità del codice 4
4. Correttezza logica 5
5. Verifica delle precondizioni 5
6. Verifica delle postcondizioni 6
7. Verifica degli invarianti 6
8. Controllo dei casi limite 7
9. Gestione degli errori 7
10. Controllo della sicurezza 8
11. Analisi delle dipendenze 8
12. Controllo della duplicazione del codice 9
13. Verifica della manutenibilità 9
14. Controllo della documentazione 10
15. Verifica dello stile di programmazione 10
16. Analisi preliminare delle prestazioni 11
17. Controllo della gestione delle risorse 11
18. Analisi della testabilità 12
19. Individuazione di codice morto o irraggiungibile 12
20. Esito finale dell'ispezione 13
5. Criticità individuate e miglioramenti consigliati 14
6. Punti di forza rilevati 15
7. Conclusione 15

1. Introduzione

Il presente documento raccoglie la nuova analisi statica e l'ispezione del codice del sistema ascensore. L'analisi considera la versione aggiornata del progetto, nella quale il nucleo logico Java/JML è rimasto separato dalla parte web dimostrativa e la pagina web è stata suddivisa in file dedicati: index.html, style.css e script.js.

Rispetto alla versione precedente, l'ispezione tiene conto anche delle modifiche ai test Selenium, ora basati su JUnit 5 e su attese esplicite tramite WebDriverWait, e della configurazione di integrazione continua tramite GitHub Actions. Queste modifiche incidono soprattutto su dipendenze, duplicazione, manutenibilità e stile di programmazione.

2. Metodo di ispezione

L'ispezione è organizzata in 20 punti di controllo. Per ogni punto sono valutati l'aspetto osservato, le evidenze presenti nel codice aggiornato, eventuali criticità residue e un esito sintetico.

La classificazione dell'esito usa tre livelli:

- Soddisfatto: il codice rispetta il criterio considerato.
- Parzialmente soddisfatto: il criterio è rispettato nel nucleo principale, ma sono presenti miglioramenti consigliati o limiti localizzati.
- Da migliorare: il criterio presenta criticità significative rispetto all'obiettivo dell'ispezione.

3. Riepilogo dei 20 punti di controllo

N.	Punto di controllo	Criterio	Esito
1	Obiettivo dell'analisi	Definizione dello scopo dell'ispezione	Soddisfatto
2	Struttura del programma	Organizzazione in classi, package e responsabilità	Soddisfatto
3	Leggibilità del codice	Nomi, chiarezza, formattazione e comprensibilità	Soddisfatto
4	Correttezza logica	Coerenza di condizioni, cicli, transizioni e flusso	Soddisfatto
5	Precondizioni	Validità degli input e delle condizioni di chiamata	Soddisfatto

6	Postcondizioni	Coerenza dello stato dopo l'esecuzione dei metodi	Soddisfatto
7	Invarianti	Proprietà sempre vere sullo stato dell'ascensore	Soddisfatto
8	Casi limite	Piani estremi, input invalidi, assenza richieste, anomalie	Soddisfatto
9	Gestione errori	Null, guasto, overload, input non validi, errori HTTP	Parzialmente soddisfatto
10	Sicurezza	Movimento sicuro, protezione stati anomali e validazione input	Soddisfatto
11	Dipendenze	Uso di librerie e componenti esterni	Soddisfatto
12	Duplicazione	Ripetizioni e possibilità di estrazione di codice comune	Soddisfatto
13	Manutenibilità	Facilità di modifica, separazione e scalabilità	Soddisfatto
14	Documentazione	Coerenza tra codice, JML e documenti descrittivi	Soddisfatto
15	Stile di programmazione	Convenzioni, ordine e uniformità	Soddisfatto
16	Prestazioni	Possibili inefficienze rilevabili staticamente	Soddisfatto
17	Gestione risorse	Stream, server, driver, thread e sincronizzazione	Parzialmente soddisfatto
18	Testabilità	Isolabilità del nucleo e copertura tramite test	Soddisfatto
19	Codice morto/irraggiungibile	Metodi, rami o variabili poco utili	Parzialmente soddisfatto
20	Esito finale	Valutazione complessiva dell'ispezione	Soddisfatto

4. Analisi dettagliata punto per punto

1. Obiettivo dell'analisi

Esito: Soddisfatto

Osservazioni:

- L'obiettivo dell'ispezione è chiaro: verificare staticamente la qualità e la correttezza del codice Java/JML rispetto al comportamento atteso del sistema ascensore.
- L'analisi considera il nucleo logico, l'interfaccia web dimostrativa, i test JUnit 5, i test Selenium e la configurazione GitHub Actions usata per l'esecuzione in CI.
- La verifica statica è distinta dai test automatici: i test mostrano che il comportamento osservato è corretto, mentre l'ispezione valuta qualità, struttura e mantenibilità del codice.

2. Struttura generale del programma

Esito: Soddisfatto

Osservazioni:

- Il progetto è organizzato in package con responsabilità distinte: progetto contiene il nucleo logico, web contiene il server HTTP, test contiene i test automatici.
- La classe Ascensore incapsula stato e operazioni fondamentali; ControlloreAscensore coordina il passo logico; InputAscensore rappresenta gli ingressi; le enum rappresentano domini finiti e riducono il rischio di valori non ammessi.
- La parte web è stata migliorata separando la pagina in index.html, style.css e script.js, mentre AscensoreHttpServer resta responsabile di routing, stato JSON, azioni e file statici.

3. Leggibilità del codice

Esito: Soddisfatto

Osservazioni:

- I nomi delle classi e dei metodi sono descrittivi e coerenti con il dominio: aggiungiRichiestaInterna, gestisciSovraccarico, gestisciTimerGuasto, scegliDirezione e muoviDiUnPiano descrivono chiaramente l'azione eseguita.
- L'uso di enum rende leggibili gli stati dell'ascensore, evitando stringhe libere o codici numerici.
- La separazione dei file web ha eliminato la lunga costruzione della pagina tramite html.append(...), rendendo più semplice leggere sia il server sia l'interfaccia.

4. Correttezza logica

Esito: Soddisfatto

Osservazioni:

- Il flusso del controllore è ordinato: acquisizione richieste, gestione persone, gestione errori e comportamento normale.
- Il movimento avviene solo dopo la scelta della direzione e solo con porte chiuse e assenza di errore.
- Il servizio del piano corrente apre le porte, ferma la cabina, annulla la direzione e rimuove la richiesta associata al piano.
- La gestione di GUASTO e OVERLOAD è separata: il guasto blocca l'acquisizione di nuove richieste, mentre l'overload mantiene le porte aperte per consentire l'uscita delle persone.

5. Verifica delle precondizioni

Esito: Soddisfatto

Osservazioni:

- I metodi principali della classe Ascensore sono protetti da precondizioni JML: validità del piano, assenza di guasto, stato delle porte e compatibilità dello stato cabina.
- Il metodo indiceDelPiano richiede un piano valido, evitando accessi fuori dai limiti dell'array.
- Il controllore esegue controlli difensivi prima di invocare metodi con precondizioni più restrittive, ad esempio ignorando richieste non valide e valori negativi per persone entrate o uscite.
- Il costruttore di ControlloreAscensore rifiuta un Ascensore nullo e il metodo eseguiPasso rifiuta InputAscensore nullo.

6. Verifica delle postcondizioni

Esito: Soddisfatto

Osservazioni:

- Le postcondizioni JML descrivono gli effetti dei metodi principali: aggiunta richieste, aggiornamento persone, overload, guasto, decremento timer, servizio piano e movimento.
- Il costruttore garantisce lo stato iniziale atteso: piano 0, timer 0, nessun errore, porte chiuse e direzione nulla.
- Le postcondizioni del movimento assicurano che il piano corrente rimanga entro i limiti e che agli estremi la cabina si fermi annullando la direzione.

Criticità o miglioramenti consigliati:

- Alcuni metodi, come scegliDirezione, hanno postcondizioni essenziali ma non completamente esaustive rispetto a tutti i rami decisionali. La copertura tramite test compensa parzialmente questa scelta.

7. Verifica degli invarianti

Esito: Soddisfatto

Osservazioni:

- La classe Ascensore definisce invarianti JML significativi su limiti numerici, non nullità, dimensione dell'array richieste e sicurezza del movimento.
- Gli invarianti vietano configurazioni pericolose come cabina in movimento con porte aperte, movimento durante guasto, timer positivo fuori dallo stato GUASTO e stati anomali incoerenti.
- Le proprietà di GUASTO e OVERLOAD sono formalizzate nello stato dell'ascensore e risultano coerenti con il comportamento implementato.

8. Controllo dei casi limite

Esito: Soddisfatto

Osservazioni:

- Il codice considera i piani estremi PIANO_MINIMO = -1 e PIANO_MASSIMO = 4.
- Le chiamate esterne sono filtrate: la salita non è ammessa al piano massimo e la discesa non è ammessa al piano minimo.
- Il movimento ai limiti impedisce di superare il piano minimo o massimo e annulla la direzione se non è possibile procedere.
- Sono gestiti input nulli, valori negativi per persone entrate/uscite, richieste non valide, assenza di richieste, timer a zero e richieste al piano corrente.

9. Gestione degli errori

Esito: Parzialmente soddisfatto

Osservazioni:

- Il nucleo logico gestisce due errori principali: OVERLOAD e GUASTO.
- In stato GUASTO il sistema blocca la cabina, chiude le porte, annulla la direzione e usa un timer di ripristino.
- In stato OVERLOAD la cabina viene bloccata, le porte rimangono aperte e il sistema permette l'uscita delle persone per risolvere l'anomalia.
- La parte HTTP restituisce errore 405 per metodi non supportati, 404 per risorse statiche mancanti e JSON di errore in caso di eccezione.

Criticità o miglioramenti consigliati:

- La parte web usa ancora valori di default per alcuni input non numerici; questo evita crash, ma può nascondere errori dell'utente trasformandoli in 0.
- La gestione degli errori HTTP è sufficiente per una demo locale, ma non distingue in modo fine tutti gli errori applicativi.

10. Controllo della sicurezza

Esito: Soddisfatto

Osservazioni:

- Dal punto di vista del modello applicativo, la sicurezza è ben rappresentata: niente movimento con porte aperte, niente movimento in guasto e blocco in sovraccarico.
- Le proprietà di sicurezza sono espresse sia negli invarianti JML sia nel flusso del controllore.
- Lo stato interno è incapsulato: gli attributi sono private e l'accesso avviene tramite metodi controllati.

Criticità o miglioramenti consigliati:

- La sicurezza informatica dell'interfaccia web è limitata perché il server è una demo locale: non sono presenti autenticazione, autorizzazione o protezioni CSRF. Per il contesto didattico locale questo è accettabile.

11. Analisi delle dipendenze

Esito: Soddisfatto

Osservazioni:

- Il nucleo logico dipende quasi solo da Java standard e dalle annotazioni JML.
- La parte web usa com.sun.net.httpserver.HttpServer, soluzione leggera e sufficiente per una dimostrazione locale senza introdurre framework esterni.
- I test sono ora coerenti con JUnit 5: gli import usano org.junit.jupiter.api e le lifecycle annotations sono BeforeAll, BeforeEach e AfterAll.
- Selenium è confinato ai test di integrazione web e non entra nel nucleo logico.
- La pipeline GitHub Actions installa Java 17, Chrome, ChromeDriver, JUnit Platform Console e costruisce il classpath Selenium in modo esplicito.

Criticità o miglioramenti consigliati:

- com.sun.net.httpserver resta una scelta dimostrativa, non una base produttiva per applicazioni web reali. Nel perimetro del progetto questa dipendenza è comunque adeguata.

12. Controllo della duplicazione del codice

Esito: Soddisfatto

Osservazioni:

- La duplicazione nel nucleo logico è contenuta: i comportamenti comuni sono centralizzati e i metodi hanno responsabilità specifiche.
- La parte web non costruisce più HTML, CSS e JavaScript tramite concatenazioni nel server: la duplicazione e il rumore della classe AscensoreHttpServer sono stati ridotti separando i file statici.
- Nel server sono stati estratti metodi dedicati per azioni di sistema, costruzione input, registrazione eventi, invio risposte e gestione file statici.
- Nei test Selenium sono stati introdotti metodi di supporto per attese, cambio modalità e verifica delle condizioni, riducendo ripetizioni e codice fragile.

Criticità o miglioramenti consigliati:

- Resta una fisiologica somiglianza tra richiestaInterna, chiamataSalita e chiamataDiscesa lato JavaScript; per la dimensione attuale del progetto è accettabile.

13. Verifica della manutenibilità

Esito: Soddisfatto

Osservazioni:

- La manutenibilità è migliorata in modo significativo rispetto alla versione precedente: AscensoreHttpServer non contiene più l'intera pagina costruita tramite StringBuilder.
- La separazione in index.html, style.css e script.js consente di modificare layout, stile e comportamento client senza intervenire sul server Java.
- Il server mantiene responsabilità più chiare: esposizione di endpoint, lettura file statici, gestione stato, azioni e simulazione automatica.
- Il nucleo logico rimane stabile e indipendente dalla parte web, quindi modifiche grafiche o di interfaccia non impattano direttamente le classi Ascensore e ControlloreAscensore.
- La presenza di test unitari e Selenium rende più sicure le modifiche future.

Criticità o miglioramenti consigliati:

- L'aggiunta di nuovi stati o nuove regole richiederebbe comunque modifiche coordinate in nucleo, interfaccia, test e documentazione.

14. Controllo della documentazione

Esito: Soddisfatto

Osservazioni:

- La documentazione progettuale descrive requisiti, proprietà, implementazione Java/JML, validazione ASM, scenari di verifica e test automatici.
- Le annotazioni JML fungono da documentazione formale interna al codice e rendono esplicite precondizioni, postcondizioni e invarianti.
- La nuova struttura web è più facile da documentare perché separa chiaramente server, HTML, CSS e JavaScript.

15. Verifica dello stile di programmazione

Esito: Soddisfatto

Osservazioni:

- Lo stile del nucleo logico è chiaro: nomi espressivi, attributi private, costanti static final e metodi con responsabilità limitate.
- La parte web è ora più pulita: HTML, CSS e JavaScript sono in file separati e il server non contiene più lunghe sequenze di stringhe HTML.
- I test Selenium sono stati aggiornati a JUnit 5 e usano WebDriverWait con ExpectedConditions, evitando l'uso sistematico di Thread.sleep.
- La rimozione dei System.out.println diagnostici dalla gestione dei file statici rende il codice più ordinato e meno rumoroso.
- L'uso di costanti per URL, content type, path e tempi di attesa migliora uniformità e leggibilità.

Criticità o miglioramenti consigliati:

- Si consiglia comunque di applicare un formatter Java uniforme all'intero progetto per evitare piccole differenze di indentazione tra file.

16. Analisi preliminare delle prestazioni

Esito: Soddisfatto

Osservazioni:

- La complessità del nucleo è molto bassa: la ricerca delle richieste scorre al massimo NUMERO_PIANI elementi, quindi il costo è trascurabile.
- La simulazione automatica usa un intervallo di 1000 ms e non risulta intensiva per CPU.
- La separazione dei file statici evita di rigenerare l'intera pagina HTML via codice Java a ogni richiesta.
- La serializzazione JSON dello stato è semplice e adeguata per una demo locale.

Criticità o miglioramenti consigliati:

- Se il numero di piani crescesse molto, si potrebbe valutare una struttura dati più specializzata; nel progetto attuale non è necessario.

17. Controllo della gestione delle risorse

Esito: Parzialmente soddisfatto

Osservazioni:

- Gli stream HTTP vengono chiusi con try-with-resources nei metodi di risposta e nei test per l'invio del reset.
- Il server HTTP può essere avviato e fermato tramite avvia e ferma.
- Il driver Selenium viene chiuso in tearDown tramite driver.quit().
- La simulazione automatica usa un flag volatile e sincronizzazione sul server per proteggere lo stato condiviso.
- I file statici vengono letti tramite Files.readAllBytes; per la dimensione ridotta dei file questa scelta è accettabile.

Criticità o miglioramenti consigliati:

- fermaSimulazioneAutomatica interrompe il thread ma non esegue join; per una chiusura più rigorosa si potrebbe attendere la terminazione con timeout.
- La porta 8080 è fissa: in contesti di test paralleli o ambienti CI complessi potrebbe essere già occupata.

18. Analisi della testabilità

Esito: Soddisfatto

Osservazioni:

- Il nucleo logico è altamente testabile perché Ascensore e ControlloreAscensore possono essere istanziati senza server web e senza dipendenze esterne.
- I test JUnit coprono stato iniziale, input validi e invalidi, movimento, richieste, porte, persone, overload, guasto, timer, ripristino e scelta direzione.
- I test Selenium verificano l'integrazione tra pagina, server e nucleo logico.
- L'uso di WebDriverWait rende i test web più stabili rispetto a Thread.sleep, soprattutto in CI.

19. Individuazione di codice morto o irraggiungibile

Esito: Parzialmente soddisfatto

Osservazioni:

- Nel nucleo logico non emergono metodi chiaramente inutilizzati: i metodi pubblici sono usati dal controllore, dai test o dalla parte web.
- I rami principali del controllore risultano coperti da test significativi.
- La separazione dei file statici ha eliminato il precedente blocco di generazione HTML dentro AscensoreHttpServer, riducendo codice poco manutenibile.

Criticità o miglioramenti consigliati:

- Alcuni rami della simulazione casuale e della gestione HTTP sono più difficili da verificare staticamente e potrebbero non essere completamente coperti dai test.
- La gestione degli input non numerici ricondotti a valori di default rende alcuni casi di errore meno osservabili.

20. Esito finale dell'ispezione

Esito: Soddisfatto

Osservazioni:

- L'ispezione statica conferma che il progetto è complessivamente coerente, leggibile e meglio organizzato rispetto alla versione precedente.
- Il nucleo Java/JML soddisfa le principali proprietà attese: validità dei piani, non negatività del numero di persone, assenza di stati nulli, movimento solo con porte chiuse, blocco in guasto, blocco in overload e ripristino tramite timer.
- Il controllore applica un flusso logico chiaro e coerente con i requisiti del sistema.
- Le modifiche a dipendenze, duplicazione, manutenibilità e stile hanno migliorato l'esito dei punti 11, 12, 13 e 15.
- Le criticità residue sono localizzate e non compromettono il comportamento principale dell'ascensore.

5. Criticità individuate e miglioramenti consigliati

Priorità	Area	Criticità	Miglioramento consigliato
Bassa	Input web	Un input non numerico può essere ricondotto al valore 0.	Validare in modo esplicito lato client e lato server, restituendo un messaggio di errore chiaro.
Bassa	Thread simulazione	Il thread viene interrotto ma non viene atteso con join.	Aggiungere join con timeout nella fase di arresto della simulazione automatica.
Bassa	Porta HTTP	La porta 8080 è fissa.	Rendere la porta configurabile, soprattutto per test paralleli o ambienti CI.
Bassa	Sicurezza web	La demo locale non implementa autenticazione o protezioni CSRF.	Mantenere la limitazione a uso locale oppure introdurre protezioni se il servizio viene esposto in rete.
Bassa	Copertura simulazione casuale	Alcuni rami casuali sono meno direttamente verificabili.	Aggiungere test con Random iniettabile o metodi separati più facilmente testabili.
Bassa	Stile	Possibili piccole differenze di indentazione tra file.	Applicare un formatter Java uniforme e una convenzione condivisa.

6. Punti di forza rilevati

- Separazione efficace tra stato dell'ascensore, input e logica di coordinamento.
- Uso di enum per rappresentare domini finiti e ridurre stati non ammessi.
- Presenza di invarianti JML significativi sulle proprietà di sicurezza.
- Gestione esplicita di guasto, sovraccarico e ripristino.
- Separazione della parte web in index.html, style.css e script.js.
- Server HTTP più leggibile, con gestione dedicata di file statici, stato JSON e azioni.
- Test JUnit 5 estesi sui casi principali e sui casi limite.
- Test Selenium più stabili grazie all'uso di WebDriverWait ed ExpectedConditions.
- Pipeline GitHub Actions configurata per Java 17, Chrome, ChromeDriver, JUnit e Selenium.
- Documentazione coerente con requisiti, implementazione e validazione del modello.

7. Conclusione

La nuova analisi statica evidenzia un progetto complessivamente solido e più maturo rispetto alla versione precedente. Il nucleo logico dell'ascensore rimane chiaro, incapsulato e coerente con i requisiti funzionali e con le proprietà di sicurezza definite. La presenza di contratti JML rafforza la specifica formale del comportamento e rende esplicite le condizioni che devono rimanere sempre valide.

Le modifiche apportate alla parte web migliorano in modo concreto la manutenibilità: la classe AscensoreHttpServer non contiene più l'intera interfaccia costruita tramite stringhe, mentre HTML, CSS e JavaScript sono gestiti come file statici separati. Questo rende più semplice modificare la grafica, il layout o il comportamento client senza toccare il nucleo Java/JML.

Anche i test sono stati migliorati: l'adozione coerente di JUnit 5 e l'eliminazione dei Thread.sleep nei test Selenium a favore di WebDriverWait rendono la suite più leggibile e più stabile, soprattutto in ambiente CI. La pipeline GitHub Actions completa il quadro, permettendo di verificare automaticamente il progetto a ogni aggiornamento.

Le criticità residue sono limitate e riguardano soprattutto aspetti non centrali per il simulatore didattico: gestione più esplicita degli input web non validi, porta HTTP configurabile, join del thread di simulazione e hardening della parte web in caso di esposizione reale. Nel complesso, l'esito dell'ispezione è positivo e i punti 11, 12, 13 e 15 risultano migliorati rispetto alla precedente analisi.